

Instituto Tecnológico de
Milpa Alta 2

Ingeniería en Sistemas
Computacionales

Séptimo Semestre

Lenguajes y Automatas 2

Unidad 1

Axel Jael Jimenez Leyte

231190061

Introducción

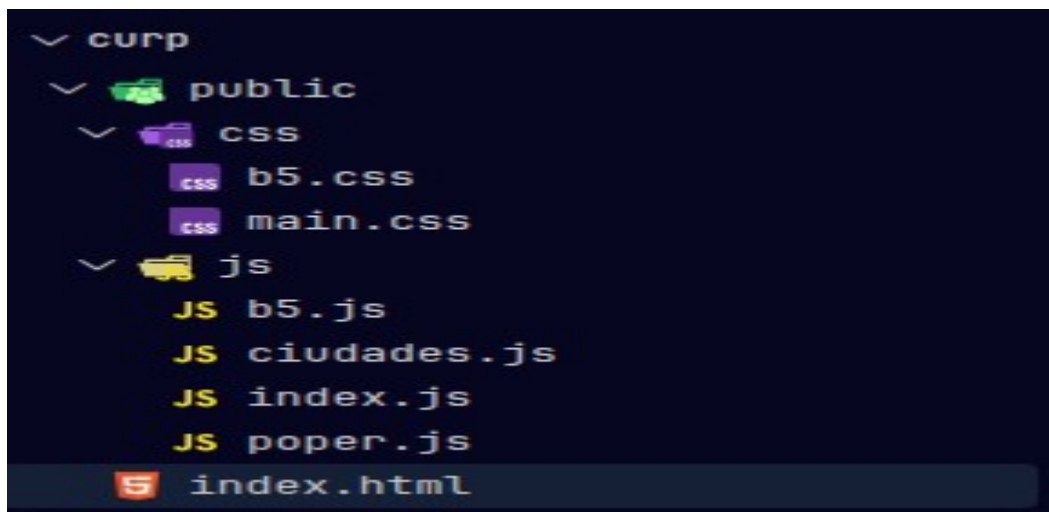
El presente proyecto consiste en el desarrollo de una aplicación web interactiva orientada a la generación y validación sintáctica de la Clave Única de Registro de Población (**CURP**) para ciudadanos mexicanos, implementando el análisis de cadenas mediante **Expresiones Regulares (RegEx)**.

Para la construcción del sistema se empleó la siguiente pila tecnológica:

- **HTML5:** Define la estructura semántica de la página y el formulario de captura.
- **Bootstrap 5 & CSS:** Provee los componentes visuales, tipografías y el diseño responsivo de la interfaz.
- **JavaScript (Vanilla JS):** Gestiona la captura de eventos en el DOM, la validación de campos obligatorios, la normalización Unicode, las transiciones léxicas con expresiones regulares y la construcción algorítmica de la CURP bajo la normativa de RENAPO.

2. Estructura del Proyecto (Árbol de Directorios)

La organización de los archivos en el entorno de trabajo se estructura de la siguiente manera:



3. Desarrollo e Implementación Técnica

Módulo 1: Estructura de Captura en HTML5 (index.html)

Descripción

El archivo index.html contiene la interfaz de usuario. Se utilizó una rejilla responsiva centrada (container, row justify-content-center) y un formulario con identificador #formulario. A cada control de entrada (input y select) se le asignó un id único para su posterior manipulación en JavaScript.

Se omitió intencionalmente el atributo nativo required del navegador en todos los campos con el fin de delegar la validación completa a la lógica de JavaScript. Al final del cuerpo se importan secuencialmente ciudades.js e index.js.

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/css/bootstrap.min.css" rel="stylesheet" integrity="sha384-EVSTQN3/azprG1Anm300p3JLw9Na08Yz1rttcQTWfSpd3y065VohhpuuCOMLASJC" crossorigin="anonymous">
7   <link rel="stylesheet" href="public/css/b5.css">
8   <link rel="stylesheet" href="main.css">
9   <script src="public/js/b5.js"></script>
10  <script src="public/js/popper.js"></script>
11  <script src="public/js/leader.js"></script>
12  <title>Generacion del CURP</title>
13 </head>
14 <body>
15   <div class="container">
16     <div class="row justify-content-center">
17       <div class="col-md-12 text-center mb-3">
18         <h1>Generacion del CURP</h1>
19       </div>
20     </div>
21     <div class="text-center">Datos Personales</div>
22     <div class="col-md-6">
23       <form id="formulario">
24         <div class="mb-3">
25           <label for="nombre" class="form-label">Nombre</label>
26           <input type="text" class="form-control" id="nombre">
27         </div>
28         <div class="mb-3">
29           <label for="apellidoPaterno" class="form-label">Apellido Paterno</label>
30           <input type="text" class="form-control" id="apellidoPaterno">
31         </div>
32         <div class="mb-3">
33           <label for="apellidoMaterno" class="form-label">Apellido Materno</label>
34           <input type="text" class="form-control" id="apellidoMaterno">
35         </div>
36         <div class="mb-3">
37           <label for="fechaNacimiento" class="form-label">Fecha de Nacimiento</label>
38           <input type="date" class="form-control" id="fechaNacimiento">
39         </div>
40         <div class="mb-3">
41           <label for="sexo" class="form-label">Sexo</label>
42           <select class="form-select" id="sexo">
43             <option value="">Seleccione</option>
44             <option value="H">Hombre</option>
45             <option value="M">Mujer</option>
46           </select>
47         </div>
48         <div class="mb-3">
49           <label for="ciudadNacimiento" class="form-label">Entidad de Nacimiento</label>
50           <select class="form-select" id="ciudadNacimiento">
51             <option value="">Seleccione una entidad</option>
52           </select>
53         </div>
54         <button type="submit" class="btn btn-primary">Generar CURP</button>
55       </form>
56     </div>
57   </div>
58 </body>
59 <script src="public/js/ciudades.js"></script>
60 <script src="public/js/index.js"></script>
61 </html>
```

Funcionamiento

- Proporciona los campos de entrada donde el usuario digita nombres, apellidos, fecha de nacimiento, sexo y entidad federativa.
- El elemento `<select id="ciudadNacimiento">` se mantiene inicialmente vacío para ser alimentado dinámicamente desde el archivo de datos.

Generacion del CURP

Datos Personales

Nombre

Apellido Paterno

Apellido Materno

Fecha de Nacimiento

dd / mm / aaaa

Sexo

Seleccione

Entidad de Nacimiento

Seleccione una entidad

Generar CURP

Módulo 2: Catálogo de Entidades Federativas (ciudades.js)

Descripción

Define una fuente de datos inmutable (`Object.freeze`) que encapsula la lista de las 32 entidades federativas de México junto con la clave para nacidos en el extranjero, asociando a cada una su nombre oficial y su clave normativa de dos caracteres alfabéticos.

Fragmento de Código

```
const obtenerCiudades = () => Object.freeze([
  { nombre: 'Aguascalientes', clave: 'AS' },
  { nombre: 'Baja California', clave: 'BC' },
  { nombre: 'Baja California Sur', clave: 'BS' },
  { nombre: 'Campeche', clave: 'CC' },
  { nombre: 'Coahuila', clave: 'CL' },
  { nombre: 'Colima', clave: 'CM' },
  { nombre: 'Chiapas', clave: 'CS' },
  { nombre: 'Chihuahua', clave: 'CH' },
  { nombre: 'Ciudad de México', clave: 'DF' },
  { nombre: 'Durango', clave: 'DG' },
  { nombre: 'Guanajuato', clave: 'GT' },
  { nombre: 'Guerrero', clave: 'GR' },
  { nombre: 'Hidalgo', clave: 'HG' },
  { nombre: 'Jalisco', clave: 'JC' },
  { nombre: 'Estado de México', clave: 'MC' },
  { nombre: 'Michoacán', clave: 'MN' },
  { nombre: 'Morelos', clave: 'MS' },
  { nombre: 'Nayarit', clave: 'NT' },
  { nombre: 'Nuevo León', clave: 'NL' },
  { nombre: 'Oaxaca', clave: 'OC' },
  { nombre: 'Puebla', clave: 'PL' },
  { nombre: 'Querétaro', clave: 'QT' },
  { nombre: 'Quintana Roo', clave: 'QR' },
  { nombre: 'San Luis Potosí', clave: 'SP' },
  { nombre: 'Sinaloa', clave: 'SL' },
  { nombre: 'Sonora', clave: 'SR' },
  { nombre: 'Tabasco', clave: 'TC' },
  { nombre: 'Tamaulipas', clave: 'TS' },
  { nombre: 'Tlaxcala', clave: 'TL' },
  { nombre: 'Veracruz', clave: 'VZ' },
  { nombre: 'Yucatán', clave: 'YN' },
  { nombre: 'Zacatecas', clave: 'ZS' },
  { nombre: 'Nacido en el extranjero', clave: 'NE' }
]);
```

Funcionamiento

- La función obtenerCiudades() devuelve la colección de objetos estructurados como par nombre y clave.
- Al usar Object.freeze, se previene cualquier mutación o sobreescritura accidental del catálogo en tiempo de ejecución.

Módulo 3: Poblado Dinámico del Selector y Mapeo del DOM (index.js)

Descripción

Al iniciar el script, se capturan las referencias a los nodos del DOM en un objeto asociativo campos y se recorre el catálogo de ciudades para inyectar cada estado como una opción interactiva dentro del menú desplegable.

Funcionamiento

- ciudades.forEach crea de forma dinámica elementos <option>, asignando el nombre del estado como valor y mostrando en pantalla la descripción junto con su clave entre paréntesis (por ejemplo: Ciudad de México (DF)).

Módulo 4: Definición de Autómatas y Expresiones Regulares

Descripción

Se definen los patrones formales que funcionarán como aceptores de lenguaje para la validación de cada tipo de entrada, así como la lista negra de palabras consideradas inconvenientes por la autoridad registral.

```
//mandamos a traer nuestra lista de ciudades con su clave-valor
const ciudades = obtenerCiudades();
//guardamos las expresiones que usaremos para validar los datos
const expresiones = {
  //validamos que el nombre tenga letras, espacios o guiones
  nombre: /^[A-ZÁÉÍÓÚÑ' -]+$/i,
  //validamos que la fecha tenga el formato de año, mes y día
  fecha: /^\d{4}-\d{2}-\d{2}$/i,
  //validamos que el sexo sea H de hombre o M de mujer
  sexo: /^[HM]$/i,
  //validamos que la curp tenga la estructura esperada
  curp: /^[A-Z]{4}\d{6}[HM][A-Z]{5}[A-Z0-9]\d$/i
};
```

Explicación de los Patrones

1. **expresiones.nombre** (/^[A-ZÁÉÍÓÚÑ' -]+\$/i): Acepta únicamente caracteres alfabéticos válidos en español, vocales con acento, diéresis, espacios y guiones. La bandera i permite indistinción entre mayúsculas y minúsculas.
2. **expresiones.fecha** (/^\d{4}-\d{2}-\d{2}\$/): Garantiza que la cadena tenga exactamente 4 dígitos de año, 2 de mes y 2 de día divididos por guiones.
3. **expresiones.sexo** (/^[HM]\$/): Lenguaje restringido a un único símbolo: H o M.
4. **expresiones.curp** (/^[A-Z]{4}\d{6}[HM][A-Z]{5}[A-Z0-9]\d\$/): Expresión de validación global que comprueba la longitud exacta de 18 caracteres y la distribución estructural de letras y dígitos.
5. **palabrasAntisonantes**: Expresión de coincidencia exacta que agrupa los términos altisonantes que no deben formarse en las primeras cuatro letras de la clave.

```
//aquí normalizamos todos los datos
const normalizar = (texto) => texto
  //separamos las letras con acentos de sus marcas
  .normalize('NFD')
  //quitamos las marcas de los acentos
  .replace(/[\u0300-\u036f]/g, '')
  //quitamos espacios al inicio y al final
  .trim()
  //convertimos todo el texto a mayusculas
  .toUpperCase();
```

Módulo 5: Normalización de Cadenas y Búsqueda de Entidad

Descripción

Se implementan utilerías para transformar el texto a una representación canónica y para localizar objetos de ciudad a partir de una cadena dada.

```

//buscamos la ciudad cuyo nombre coincida con el recibido
const buscarCiudad = (nombre) => {
  //recorremos la lista de ciudades una por una
  for (let indice = 0; indice < ciudades.length; indice++) {
    //obtenemos la ciudad actual
    let ciudad = ciudades[indice];
    //normalizamos ambos nombres antes de compararlos
    let nombreCiudad = normalizar(ciudad.nombre);
    let nombreBuscado = normalizar(nombre);
    //si los nombres coinciden devolvemos la ciudad
    if (nombreCiudad === nombreBuscado) {
      return ciudad;
    }
  }
  //si no encontramos la ciudad devolvemos null
  return null;
};

```

Funcionamiento

- .normalize('NFD') separa los caracteres base de sus signos diacríticos (acentos, diéresis).
- .replace(/[\u0300-\u036f]/g, '') elimina el bloque Unicode de diacríticos, convirtiendo por ejemplo Á en A y é en e.
- trim() y toUpperCase() remueven espacios residuales y estandarizan la cadena en mayúsculas.
- buscarCiudad() recorre el arreglo comparando nombres previamente normalizados para evitar discrepancias ortográficas.

Módulo 6: Reglas Léxicas de Extracción y Tratamiento Especial

Descripción

Funciones encargadas de extraer los caracteres correspondientes a las posiciones internas del CURP (primera vocal interna y primera consonante interna), así como el tratamiento de nombres de pila comunes y palabras inconvenientes.

Funcionamiento

- **obtenerVocal y obtenerConsonante:** Usan `.slice(1)` para omitir la letra inicial del apellido o nombre y ejecutan `.match()` con una expresión regular para encontrar el primer carácter que pertenezca al conjunto especificado; si no existe, devuelven 'X'.
- **obtenerPrimerNombre:** Evalúa con `/^(MARIA|MA|JOSE|J)$/` si el primer nombre es común; de existir un segundo nombre compuesto, selecciona este último conforme a la regla del registro civil.
- **corregirPalabraAntisonante:** Si las 4 primeras letras coinciden con el patrón de palabras no deseadas, sustituye la vocal interna (posición índice 1) por el carácter 'X'.

[INSERTAR AQUÍ CAPTURA 7: Bloque de funciones de extracción de caracteres en el código fuente]

Módulo 7: Algoritmo de Armado y Generación de Homoclave

Descripción

Concatena los segmentos calculados para formar los 16 caracteres base del CURP y calcula un sufijo diferenciador (homoclave) mediante un algoritmo de sumatoria de códigos ASCII ponderados por posición.

```

//buscamos la ciudad cuyo nombre coincida con el recibido
const buscarCiudad = (nombre) => {
  //recorremos la lista de ciudades una por una
  for (let indice = 0; indice < ciudades.length; indice++) {
    //obtenemos la ciudad actual
    let ciudad = ciudades[indice];
    //normalizamos ambos nombres antes de compararlos
    let nombreCiudad = normalizar(ciudad.nombre);
    let nombreBuscado = normalizar(nombre);
    //si los nombres coinciden devolvemos la ciudad
    if (nombreCiudad === nombreBuscado) {
      return ciudad;
    }
  }
  //si no encontramos la ciudad devolvemos null
  return null;
};

//comprobamos si la ciudad existe
const validarCiudad = (nombre) => {
  //buscamos la ciudad recibida
  let ciudad = buscarCiudad(nombre);
  //si existe devolvemos verdadero, si no devolvemos falso
  if (ciudad !== null) {
    return true;
  }
  return false;
};

//obtenemos la primera vocal del apellido despues de su primera letra
const obtenerVocal = (apellido) => {
  //separamos el apellido desde la segunda letra
  let resto = apellido.slice(1);
  //buscamos una vocal dentro del resto del apellido
  let vocal = resto.match(/[AEIOU]/);
  //si encontramos una vocal la devolvemos
  if (vocal) {
    return vocal[0];
  }
  //si no encontramos una vocal devolvemos X
  return 'X';
};

//obtenemos la primera consonante del texto despues de su primera letra
const obtenerConsonante = (texto) => {

```

Funcionamiento

1. Extrae año (AAMMDD), sexo y clave de la entidad.
2. Integra la primera letra y vocal interna del apellido paterno, inicial del materno e inicial del nombre.
3. Incorpora las tres consonantes internas.
4. Aplica la corrección antisonante.
5. Calcula y anexa los dos últimos dígitos (letra calculada por residuo módulo 26 y dígito final por residuo módulo 10).

Módulo 8: Validación de Formulario en JavaScript y Despliegue de Resultados

Descripción

Al procesar el evento submit, se aplica un esquema de validación en dos capas:

1. **Primera capa (Campos vacíos):** Revisa que ningún campo esté vacío o compuesto únicamente por espacios en blanco, alertando de inmediato si falta información.

2. **Segunda capa (Sintaxis y Lógica):** Valida formato de texto con expresiones regulares, comprueba la existencia de la fecha real mediante el objeto Date (incluyendo control de años bisiestos y fechas no futuras) y verifica la entidad federativa.

Si los datos son correctos, se invoca `mostrarResultado()` para insertar el resultado en la interfaz.

```

//escuchamos el envio del formulario
formulario.addEventListener('submit', (evento) => {
  //evitamos que la pagina se recargue al enviar el formulario
  evento.preventDefault();
  //comprobamos si algun campo esta vacio
  let camposVacios = false;
  //revisamos los campos de texto
  if (campos.nombre.value.trim() === '' ||
      campos.apellidoPaterno.value.trim() === '' ||
      campos.apellidoMaterno.value.trim() === '' ||
      campos.fechaNacimiento.value === '' ||
      campos.sexo.value === '' ||
      campos.ciudadNacimiento.value === '') {
    //indicamos que hay por lo menos un campo vacio
    camposVacios = true;
  }
  //si hay campos vacios detenemos el proceso
  if (camposVacios) {
    //avisamos al usuario que debe llenar todos los campos
    alert('Completa todos los campos antes de generar la CURP');
    //detenemos el envio del formulario
    return;
  }
  //creamos un objeto con todos los datos normalizados del form
  const datos = {
    //normalizamos el nombre escrito por la persona
    nombre: normalizar(campos.nombre.value),
    //normalizamos el apellido paterno
    apellidoPaterno: normalizar(campos.apellidoPaterno.value),
    //normalizamos el apellido materno
    apellidoMaterno: normalizar(campos.apellidoMaterno.value),
    //guardamos la fecha seleccionada
    fecha: campos.fechaNacimiento.value,
    //guardamos el sexo seleccionado
    sexo: campos.sexo.value,
    //buscamos y guardamos el objeto de la ciudad seleccionada
    ciudad: buscarCiudad(campos.ciudadNacimiento.value)
  };
  //validamos que los tres nombres o apellidos tengan un formato
  let nombresValidos = true;
  //validamos el nombre
  if (!validarNombre(datos.nombre)) {

```

Funcionamiento

- Detiene el refresco nativo del navegador con evento.preventDefault().
- Si faltan campos, emite la alerta preventiva y suspende la ejecución (return).
- Si un dato no pasa la expresión regular, activa los estilos visuales de Bootstrap agregando la clase was-validated.
- Al aprobarse todas las validaciones, genera de manera dinámica un elemento <output> con la clase visual alert alert-success que muestra la CURP final en pantalla.

Generacion del CURP

Datos Personales

Nombre

Apellido Paterno

Apellido Materno

Fecha de Nacimiento



Sexo



Entidad de Nacimiento



Generar CURP

CURP generada: JILA051207HDFMYXW0

5. Conclusiones

La realización de esta práctica demostró la utilidad práctica de los conceptos de la materia de **Lenguajes y Autómatas** aplicados al desarrollo web:

1. Las expresiones regulares actúan como autómatas finitos deterministas capaces de reconocer y validar patrones de lenguaje definidos formalmente, asegurando que las cadenas ingresadas cumplan con especificaciones sintácticas exactas antes de ser procesadas.
2. La transferencia de la validación de obligatoriedad desde el atributo HTML hacia el script de JavaScript otorga control total sobre el ciclo de vida del formulario, permitiendo encadenar validaciones por niveles (presencia de datos, formato léxico y lógica de calendario).
3. La combinación de normalización Unicode, extracción de subcadenas y comprobación mediante expresiones regulares permitió resolver de forma limpia y eficiente las reglas operativas de RENAPO para el cálculo de la CURP.